F. Modbus Addressing on the M580 and Productivity2000

Appendix F of the EFC Gases & Advanced Materials Controls Standard, Revision I, October 14, 2026. Standalone copy; see the full document for the other chapters and appendices.

This appendix explains how Modbus works, how to set up Modbus addressing on our two PLC platforms — the Modicon M580 (Control Expert Classic) and the AutomationDirect Productivity2000 (Productivity Suite) — and how to match those addresses in Ignition. Software versions are listed in Section 7.1.1 of the Controls Standard.

When Modbus is used: EFC's standards are OPC UA between the M580 and SCADA and EtherNet/IP for device communications (Section 8.2 of the Controls Standard). Modbus is used for the backup and conversion cases: a device without EtherNet/IP, an EtherNet/IP option that is not available, an existing Modbus RTU system being converted to Ethernet, and Productivity2000 SCADA links (which have no OPC UA server).

Scope: Modbus TCP, Modbus RTU where a device requires it, and Modbus/TCP Security.

F.1 Modbus Basics

Modbus is a request/response protocol: a client asks and a server answers. Servers never send data on their own, so every value SCADA shows was polled. All data lives in four tables.

Table	Size	Access	Legacy prefix	Read FC	Write FC
Coils	1 bit	Read/write	0xxxx	01	05 single, 15 multiple
Discrete inputs	1 bit	Read-only	1xxxx	02	—
Input registers	16-bit word	Read-only	3xxxx	04	—
Holding registers	16-bit word	Read/write	4xxxx	03	06 single, 16 multiple

Function codes in common use

FC	Name	Per-request limit
01 / 02	Read coils / discrete inputs	2000 bits
03 / 04	Read holding / input registers	125 registers
05 / 06	Write single coil / register	1
15	Write multiple coils	1968 bits
16	Write multiple registers	123 registers
22	Mask write register (set bits inside a word)	1

FC	Name	Per-request limit
23	Read/write multiple registers in one transaction	125 read / 121 write
43/14	Read device identification	—

Framing

- **Modbus RTU (serial):** binary frames on RS-485/RS-232 with a slave address (1–247) and CRC. One master per bus.
- **Modbus TCP:** the same message with a 7-byte header (transaction ID, protocol ID, length, unit ID) on TCP port 502. No CRC.
- **RTU over TCP:** raw RTU frames (with CRC) over a TCP socket, common on low-cost serial-to-Ethernet converters. Not the same as Modbus TCP; the client driver must match.
- **Unit ID:** ignored by most native TCP devices (use 255 or 1). Through a gateway it selects the serial slave address.

Exception codes

Code	Name	Usual cause
01	Illegal function	Device does not support that FC (for example FC16 or FC22)
02	Illegal data address	Off-by-one, or a request spanning an undefined address
03	Illegal data value	Value out of range, or quantity above the device limit
04	Server device failure	Internal device fault
06	Server busy	Long command in progress; retry
0A / 0B	Gateway path / target failed	Gateway cannot reach the serial device: unit ID, wiring, baud

F.2 Addressing and Data Types

Most Modbus problems are an off-by-one address or a swapped word order. Settle both in writing for every device before building tags.

Address notation

The protocol itself only carries a 0-based offset (0–65535) within a table. Documentation adds other numbering on top:

Notation	First holding register	Where it is used
Offset (0-based)	0	Wire captures, Modicon %MW numbers, Ignition with zero-based on
Register number (1-based)	1	Most device manuals, Ignition default
Legacy 5-digit	40001	Older manuals; the leading 4 names the table

Notation	First holding register	Where it is used
Extended 6-digit	400001	Devices with more than 9999 registers; Productivity Suite

40001, 400001, 1-based HR1, and %MW0 all point to the same register (offset 0). The conversion is:

$$\text{offset} = \text{register number} - 1 = \text{legacy address} - 40001 = \text{6-digit address} - 400001$$

Confirm any vendor map by reading a known value before building the full tag set (Modbus Application Protocol Specification).

Data types and word order

- Modbus only defines bits and 16-bit words. INT/UINT use one register; DINT, UDINT, and REAL use two; LREAL and LINT use four; strings use one register per two characters.
- Bytes within a register are big-endian. The order of the two words in a 32-bit value is not standardized: high word first (ABCD) or low word first (CDAB).
- **The Modicon M580 stores 32-bit values low word first** in consecutive %MW. Ignition must have *Reverse Word Order* on for these devices.
- Test with known values: 123.456 in a REAL and 70000 in a DINT. A word swap shows up as an obviously wrong number.

F.3 Platform Comparison

	Modicon M580 (Control Expert Classic)	Productivity2000 (Productivity Suite)
Where Modbus addresses come from	You assign them: located variables at fixed %MW / %M addresses	Assigned per tag in the Tag Database (MOD Start column), automatically or by hand
Address numbering	%MWn = offset n (0-based)	6-digit, 1-based (e.g., 400001) as shown in the Tag Database
32-bit word order	Low word first	Confirm with a test value (see Section F.5)
Modbus TCP server	Built into the CPU, port 502	Built into the CPU, port 502; enable on the Ethernet port
Modbus RTU	Through an X80 serial module (BMXNOM0200) or a gateway	Built-in serial ports
PLC as Modbus client	I/O scanning (DTM Browser) or READ_VAR / WRITE_VAR	MRX (read) and MWX (write) instructions
Ignition settings	Zero-based on, Reverse Word Order on	Zero-based off (1-based); set word order to match the test value

F.4 Step-by-Step: Modicon M580 in Control Expert Classic

The M580 is a Modbus TCP server on port 502 and can also act as a client. Only **located** variables (%M, %MW) can be read over Modbus; unlocated variables and DDT instances cannot. These steps are written for Control Expert Classic v16.2.

Setting up the M580 as a Modbus server

1. **Size the located memory.** In the Project Browser, open the CPU in the PLC bus configuration and go to its *Configuration* tab. Set the number of %M bits and %MW words large enough for the full SCADA map plus room to grow (see the register map layout in Section F.9).

2. **Declare located variables.** In the Data Editor, create each variable SCADA needs and enter its address in the *Address* column, for example `PT101_PV` (REAL) at `%MW200`. Put REAL, DINT, and UDINT values on even addresses and never locate anything on the second word of a 32-bit value.

3. **Copy between your DDTs and the map.** DDT instances (DDT_Process_Value, DDT_Device, and so on) can't be read over Modbus directly. In a dedicated section of the program, copy status values from the DDTs into the located map each scan, and copy commands and setpoints from the map back into the DDTs after checking limits.

4. **Lock down access.** On the CPU's *Security* tab, enable the access control list and list only the Ignition servers and engineering stations allowed to connect. Enforce Security disables FTP, TFTP, HTTP, EtherNet/IP, SNMP, and DHCP/BOOTP; re-enable only what the project needs.

5. **Test.** Write known test values into a REAL and a DINT, then read them with a standalone Modbus client (Modbus Poll, QModMaster) before setting up Ignition.

6. **Ignition device settings.** Port 502, Zero-based Addressing **on**, Reverse Word Order **on**, timeout 2000 ms, 1 concurrent request.

Do not use FC04 (input registers) against the M580. Its behavior depends on the %IW configuration and changed in Control Expert 15.0. Use FC03 (holding registers).

The M580 as a Modbus client

Method	Setup	Function codes	Use for
I/O scanning	DTM Browser: add a generic Modbus device and define request lines with a repetition rate	FC03, 16, 23 only	Cyclic exchange with drives, meters, remote PLCs
Explicit messaging	READ_VAR / WRITE_VAR with ADDM, or DATA_EXCH	Coils, holding and input registers	Input registers, coils, occasional reads/writes

Both methods can run against the same device at the same time. Wrap explicit messaging in a standard DFB with timeout, retry, and diagnostics rather than hand-building each call. The M580 CPU has no serial Modbus port: use an X80 serial module (BMXNOM0200) or an external TCP-to-RTU gateway (Section 6.4.5 of the Controls Standard).

F.5 Step-by-Step: Productivity2000 in Productivity Suite

The P2-622 CPU has a built-in Modbus TCP server and Modbus RTU on its serial ports. In Productivity Suite, Modbus addresses belong to tags.

1. **Enable the server.** In Hardware Configuration, open the CPU's Ethernet port settings and make sure the Modbus TCP server is enabled on port 502. For Modbus RTU, set the serial port's protocol to Modbus and set the address, baud rate, and parity to match the bus.

2. **Assign Modbus addresses to tags.** Open the Tag Database. Select the tags SCADA needs, right-click the *MOD Start* column, and choose *Auto Assign Modbus Address*; or type the address in yourself. Addresses appear in 6-digit, 1-based form (e.g., `400101`).

3. **Keep the map stable.** Assign addresses deliberately, in blocks that follow the register map layout in Section F.9, so adding tags later doesn't shift existing addresses. Export the Tag Database to CSV and keep it with the project as the register map.

4. **Check 32-bit word order.** Put known values in a 32-bit integer and a float tag and read them with a Modbus client. Set Ignition's Reverse Word Order to whichever setting gives the correct values, and record the result in the register map.

5. **The P2 as a Modbus client.** Use the *MRX* (Modbus Read) and *MWX* (Modbus Write) instructions to read from or write to other Modbus devices, such as a meter or a VFD. Set the IP address (or serial port and slave address), function code, starting address, and word-swap option in the instruction. Trigger them on a timer and check the instruction's status bits for errors.

6. **Ignition device settings.** Port 502, Zero-based Addressing **off** (the P2 uses 1-based numbers), Reverse Word Order set from the test above.

[Verify the exact menu names and word-order behavior against the Productivity Suite v4.7 help files on the first project, and update this section.]

F.6 Worked Example

A pressure reading `PT101` (REAL, 32-bit) is read by Ignition from each platform.

Step	Modicon M580	Productivity2000
PLC location	`PT101_PV` located at `%MW200`–`%MW201`	Tag `PT101_PV`, MOD Start `400201` (registers 400201–400202)
Protocol offset	200	200
Ignition device	Zero-based on, Reverse Word Order on	Zero-based off, word order per test
Ignition tag path	`[M580_R23]HRF200`	`[P2_Skid1]HRF201`

Both tags read the same offset (200). The M580 path uses 200 because zero-based addressing is on; the Productivity path uses 201 because Ignition is set to 1-based numbering for that device.

F.7 Ignition Settings Reference

Perspective does not talk to Modbus directly. The Ignition Gateway's OPC UA module runs the Modbus driver as a device connection; OPC tags reference that device and Perspective views bind to the tags. On 8.1, reach serial devices through a TCP gateway.

Setting	Default	Function	M580 value
Port	502	Modbus TCP port	502
Communication Timeout	2000 ms *	Response wait	2000 ms
Concurrent Requests	1 *	Requests in flight	1–2
Max Holding Registers per Request	125	Read size (8.3.1+: separate write, 123)	125 / 123
Zero-based Addressing	Off *	On: HR0 = offset 0	**On**
Reverse Word Order	Off *	Swaps words of 32/64-bit values	**On**
Span Gaps	On *	Reads across unused addresses	On if map contiguous
Reverse String Byte Order	Off	Swaps bytes in string words	Test per device

[Defaults to be verified on the company Gateway.]*

Address syntax. Item paths take the form `[Device]HRF200`. Designators: `C/DI` (bits), `HR/IR` (INT), `HRUS` (UINT), `HRI/HRUI` (DINT/UDINT), `HRF` (REAL), `HRD` (LREAL), `HRS`*n*:*len* (string). A bit in a register is `HR100.3` (requires FC22). A unit ID is prefixed as `[GW1]3.HR100`.

M580 to Ignition cross-reference

With Zero-based Addressing and Reverse Word Order both on, the Ignition address number equals the %MW number.

M580 variable	Type	Offset	Legacy	Ignition (0-based)	Ignition (1-based)
%MW100	INT	100	400101	`HR100`	`HR101`
%MW101	UINT	101	400102	`HRUS101`	`HRUS102`
%MW200–201	REAL	200	400201	`HRF200`	`HRF201`
%MW202–203	DINT	202	400203	`HRI202`	`HRI203`
%MW300.3	BOOL in word	300	400301	`HR300.3`	`HR301.3`
%M50	EBOOL	50	000051	`C50`	`C51`
%MW400–409	STRING[20]	400	400401	`HRS400:20`	`HRS401:20`

[Confirm whether Modicon strings require Reverse String Byte Order on the first project (R23).]

F.8 Modbus TCP in Practice

- Performance depends on the number of requests needed to cover the data. Design maps so the client reads a few large contiguous blocks (up to 125 registers per read, 123 per write).
- Reading across unused addresses (span gaps) saves requests, but some devices return exception 02 for any undefined address in the block.
- Embedded devices often accept few simultaneous clients. Count SCADA servers, redundant servers, historians, laptops, and other PLCs before adding a client.

- Poll tiers: 250–500 ms for operator-critical status, 1 s for analogs, 5–30 s for configuration and diagnostics.
- Serial gateways are as slow as the serial bus. Use 19,200 baud or higher and keep gateway devices on their own driver connection.
- **Security:** Modbus TCP has no authentication or encryption; any host that can reach port 502 can read and write every register. Modbus/TCP Security (TLS on port 802) exists but device support is limited, so protection relies on network segmentation, controller IP allow-lists, and firewall rules (Chapter 8).

F.9 Company Rules

Device connections

- One Ignition device per PLC, named after the PLC tag on the System Layout drawing (e.g., `M580_R23`).
- Every M580 device: Zero-based Addressing on, Reverse Word Order on, port 502, timeout 2000 ms, concurrent requests 1. These are fixed by this standard, not chosen per project.

Register map layout

The SCADA map is a dedicated, contiguous block, separate from PLC working memory. The ranges below are %MW numbers on the M580; on the Productivity2000 use the same offsets (add 400001 for the 6-digit address).

Range	Area	Written by	Contents
0–99	System	PLC	Heartbeat, PLC mode, first-out alarm, map version, comm status of scanned devices
100–999	Status	PLC	Fixed-size blocks per UDT instance
1000–1499	Setpoints	SCADA and PLC	Setpoints and alarm limits, REAL, engineering units
1500–1999	Commands	SCADA	Command words, cleared by the PLC after action
2000+	Spare	Either	Reserved growth

- 32-bit values start on even offsets; nothing overlaps the second word of a 32-bit value.
- Analogs are REAL in engineering units. Scaled integers only where a third-party device requires them, with the factor documented.
- The map version word is incremented on every layout change; Ignition alarms on a mismatch.

[Decide fixed block sizes for DDT_Process_Value and DDT_Device (e.g., 16 and 8 words) so Ignition UDTs need only one base-address parameter.]

Commands and writes

- SCADA never writes directly to an output or interlock. It writes a request; the PLC checks permissives and interlocks (DDT_Interlock) and acts.

- One command word per device (bit 0 start, bit 1 stop, bit 2 reset, ...). The PLC clears the word after processing it.
- Setpoint writes are clamped in the PLC to engineering limits; out-of-range values are rejected and flagged.

Heartbeat and communication health

- The PLC increments a heartbeat word every second; Ignition alarms if it is unchanged for 10 s.
- Ignition writes a watchdog word every 5 s; the PLC alarms and puts remote commands in a defined safe state if it goes stale.
- Every Modbus device scanned by the PLC has a comm-healthy bit in the System area, alarmed in SCADA.

Third-party devices

Vendors and panel builders supply a register map spreadsheet with, for every point: name, table, address, **whether it is 0- or 1-based**, data type, word order, units and scaling, read/write, and poll tier. The header states supported function codes, maximum registers per request, connection limit, and unit ID. This is a Stage 1 input alongside the Standard Hardware Spreadsheet.

Security

- Modbus stays inside the control-network zone. Firewall rules allow port 502 only from named Ignition servers and engineering stations.
- The M580 access control list is enabled on every project and lists those hosts explicitly.
- No Modbus over remote-access or vendor VPN paths.

[Decide whether to trial Modbus/TCP Security or a BMENUA0100 OPC UA module on a future project.]

F.10 Troubleshooting

Symptom	Likely cause	Check / fix
Device stays "Connecting"	Route, IP, firewall, or M580 allow-list	Ping; test port 502 from the Gateway; check the CPU Security tab
All tags Bad	Address outside map, or time-outs	Gateway log exception code
Exception 01	FC not supported	Disable write-multiple or bit writes for the device
Exception 02	Off-by-one or spanned gap	Toggle zero-based to confirm; disable Span Gaps
Values shifted by one	Zero-based mismatch	Fix at the device setting
REAL wrong magnitude	Word order, or HR used for REAL	Use HRF; check Reverse Word Order
Strings pairwise swapped	Byte order in word	Toggle Reverse String Byte Order
Intermittent Bad quality	Too many requests or clients	Consolidate blocks, slow tag groups, lengthen timeout

Symptom	Likely cause	Check / fix
Exception 0A / 0B	Gateway cannot reach slave	Unit ID, RS-485 wiring, baud/parity
FC04 returns inputs on M580	Control Expert 15.0 behavior	Use FC03

Tools: Wireshark (Modbus/TCP dissector) on a mirrored port; a standalone Modbus client (e.g., Modbus Poll, QModMaster) to test the device independently of Ignition.

F.11 Sources

- Modbus Organization, Modbus Application Protocol Specification v1.1b3 and Modbus/TCP Security v36 (2021)
- Inductive Automation, Ignition 8.1 Modbus driver, Modbus addressing, 8.3 device settings
- Schneider Electric FAQs: M580 Modbus client methods, FC04 behavior, M580 Security tab
- Cybus: Connecting a Modicon M580 (located variables, addressing, and word order)
- AutomationDirect, Productivity Suite — What's New; Productivity Suite built-in help (Tag Database, MRX/MWX instructions)
- PLCtalk: Productivity Suite Modbus address auto-assignment